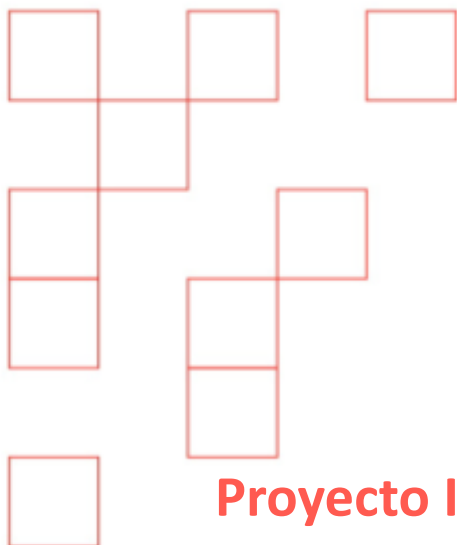
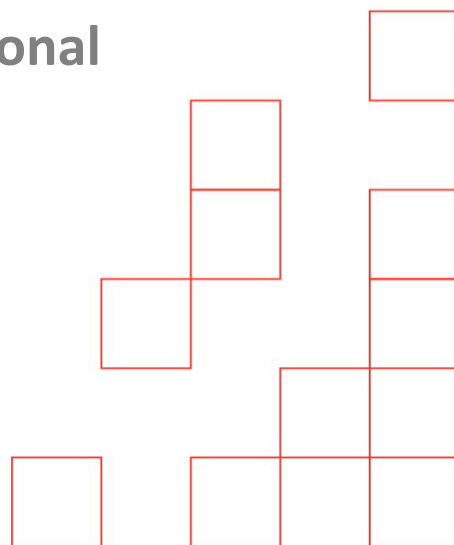




Proyecto Integrador: Autogestión

Documento Funcional

1ºDAM



Luis Daniel López Milicia.
Daniel Parra Segovia.
Hugo García Salazar.

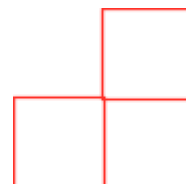
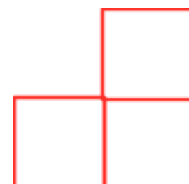


Tabla de contenido

I. Introducción.....	3
Objetivo del documento:.....	3
Alcance del proyecto:.....	3
Descripción general de la aplicación:.....	3
II. Descripción del sistema.....	4
Nombre de la aplicación:.....	4
Contexto (tienda, concesionario, librería, etc.).....	4
Usuarios objetivo:.....	4
1. Usuario No Registrado.....	4
2. Trabajador (Personal de Ventas y Gestión).....	4
3. Administrador (Gestor del Sistema).....	4
III. Requisitos funcionales.....	5
IV. Requisitos no funcionales.....	5
V. Casos de uso.....	6
VI. Diagrama de clases.....	13
VII. Modelo de datos.....	18
1. Cliente.....	18
2. Modelo.....	18
3. Trabajador.....	18
4. Vehículo.....	18
5. Venta.....	19
VIII. Arquitectura de la aplicación.....	22
Explicación del patrón MVC.....	22
Explicación de las tecnologías usadas.....	22
IX. Equipo.....	24
X. Observaciones.....	24





I. Introducción

Objetivo del documento:

El objetivo del documento es especificar y documentar los requisitos funcionales y el diseño conceptual de la aplicación de gestión de inventario para un concesionario de vehículos.

El sistema se desarrollará en el lenguaje de programación Java siguiendo el modelo MVC (modelo, vista, control). Para garantizar la persistencia de los datos la aplicación utilizará una base de datos en SQLite.

Alcance del proyecto:

El proyecto comprende el diseño y desarrollo de una aplicación de escritorio orientada a la gestión del inventario de un concesionario. El alcance incluye:

- El diseño e implementación de la base de datos.
- El desarrollo de la interfaz gráfica de usuario.
- La conexión de la base de datos con la aplicación en Java.
- La funcionalidad del controlador de la aplicación.

Descripción general de la aplicación:

Esta aplicación es un administrador de inventario de un concesionario de vehículos, donde los clientes pueden ver el catálogo y comprar vehículos, los trabajadores pueden administrar los vehículos a la venta y los modelos disponibles y los administradores pueden administrar los trabajadores del concesionario.

II. Descripción del sistema

Nombre de la aplicación:

Autogestión

Contexto (tienda, concesionario, librería, etc.)

Concesionario de vehículos.

Usuarios objetivo:

1. Usuario No Registrado

Este perfil representa a cualquier persona externa que accede al sistema público del concesionario

- Casos de uso asociados:
 - Ver catálogo
 - Comprar vehículo
 - Iniciar sesión como trabajador

2. Trabajador (Personal de Ventas y Gestión)

Este perfil corresponde a los empleados del concesionario: controlan el stock y gestionan los detalles técnicos de los vehículos.

- Casos de uso asociados:
 - Ver catálogo
 - Registrar Vehículo
 - Modificar Vehículo
 - Eliminar Vehículo
 - Registrar Marca y Modelo
 - Consultar Marca y Modelo
 - Eliminar Marca y Modelo

3. Administrador (Gestor del Sistema)

Este perfil cuenta con un rol de supervisión, además de sus funciones exclusivas de gestión de personal, tiene acceso completo a todas las herramientas operativas del trabajador.

- Casos de uso asociados:
 - Todos los del Trabajador
 - Registrar Trabajador
 - Eliminar Trabajador
 - Ver lista de trabajadores

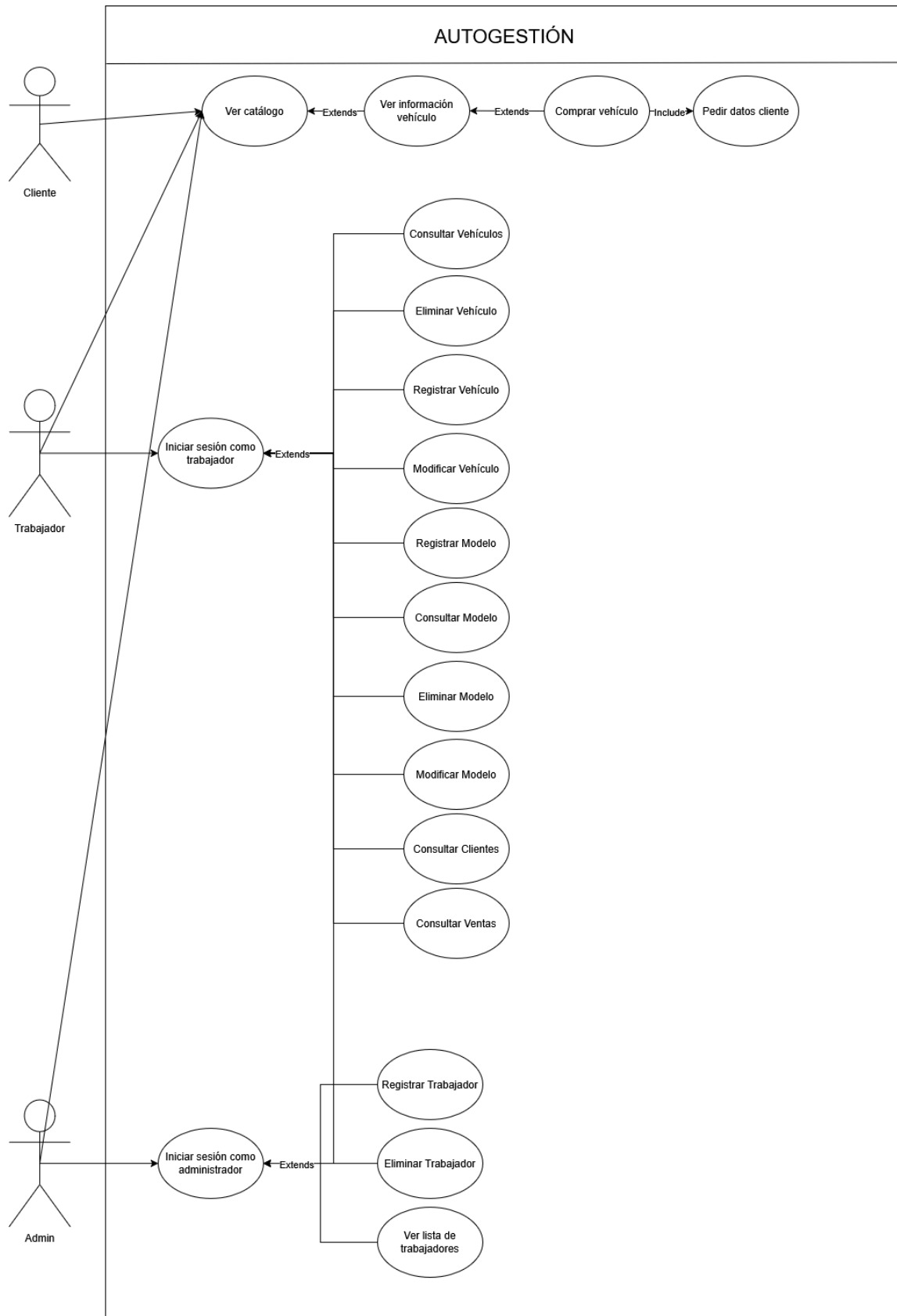
III. Requisitos funcionales

- **RF1:** Se puede registrar un vehículo al catálogo
- **RF2:** Se puede modificar un vehículo en el catálogo
- **RF3:** Se puede eliminar un vehículo del catálogo
- **RF4:** Se puede consultar el catálogo
- **RF5:** Se puede añadir marca/modelo
- **RF6:** Se puede eliminar marca/modelo
- **RF7:** Se puede consultar marca/modelo
- **RF8:** Se puede iniciar sesión como trabajador
- **RF9:** Se puede añadir un trabajador nuevo
- **RF10:** Se puede consultar la lista de trabajadores
- **RF11:** Se puede eliminar un trabajador
- **RF12:** Se puede comprar un vehículo

IV. Requisitos no funcionales

- **RNF1:** Interfaz gráfica intuitiva
- **RNF2:** Tiempo de respuesta aceptable
- **RNF3:** Persistencia en base de datos relacional
- **RNF4:** Seguridad básica (validación de datos)

V. Casos de uso





Caso de uso 1: Ver catálogo

- Actores: Cliente, Trabajador, Administrador
- Descripción: Permite visualizar el catálogo completo de vehículos disponibles.
- Flujo principal:

El actor accede a la opción Ver catálogo.

El sistema muestra la lista de vehículos.

El actor puede seleccionar un vehículo para ver su información detallada.

- Flujo alternativo:

No hay vehículos disponibles : mensaje “No se encontraron vehículos”.

Caso de uso 2: Ver información vehículo

- Actores: Cliente, Trabajador, Administrador
- Descripción: Permite consultar la información detallada de un vehículo específico (extiende de Ver catálogo).
- Flujo principal:

El actor selecciona un vehículo desde el catálogo.

El sistema muestra todos los datos del vehículo.

Según el actor se muestran las opciones correspondientes.

- Flujo alternativo:

Vehículo no encontrado o dado de baja : mensaje de error.



Caso de uso 3: Comprar vehículo

- Actor: Cliente
- Descripción: Proceso de compra de un vehículo (incluye Pedir datos cliente).
- Flujo principal:

El cliente selecciona un vehículo.

El sistema solicita los datos del cliente.

El cliente ingresa o confirma sus datos.

El sistema registra la venta y actualiza el stock.

Se genera el comprobante de compra.

- Flujo alternativo:

Datos del cliente incorrectos o incompletos : mensaje de error y retorno al ingreso de datos.

Caso de uso 4: Iniciar sesión como trabajador

- Actor: Trabajador
- Descripción: Autenticación del trabajador para acceder al sistema.
- Flujo principal:

El trabajador ingresa usuario y contraseña.

El sistema valida las credenciales.

Se concede acceso al menú de trabajador.

- Flujo alternativo:

Credenciales incorrectas : mensaje de error.



Caso de uso 5: Consultar Vehículos

- Actor: Trabajador
- Descripción: Permite buscar y visualizar la lista de vehículos registrados.
- Flujo principal:

Trabajador selecciona Consultar Vehículos.

El sistema muestra la lista de vehículos.

Caso de uso 6: Registrar Vehículo

- Actor: Trabajador
- Descripción: Añadir un nuevo vehículo al inventario.
- Flujo principal:

Trabajador ingresa los datos del vehículo.

El sistema valida la información (incluyendo que el modelo no esté vacío).

Se guarda en la base de datos.

- Flujo alternativo:

Datos inválidos : mensaje de error.

Caso de uso 7: Modificar Vehículo

- Actor: Trabajador
- Descripción: Actualizar información de un vehículo existente.
- Flujo principal:

Trabajador busca y selecciona el vehículo.

Modifica los campos deseados.



El sistema valida nuevamente los datos del vehículo.

Se guardan los cambios.

- Flujo alternativo:

Datos inválidos : mensaje de error.

Caso de uso 8: Eliminar Vehículo

- Actor: Trabajador
- Descripción: Eliminar un vehículo del sistema.
- Flujo principal:

Trabajador selecciona el vehículo.

Confirma la eliminación.

El sistema elimina o marca como eliminado.

Caso de uso 9: Registrar Modelo

- Actor: Trabajador
- Descripción: Añadir un nuevo modelo de vehículo al catálogo maestro.
- Flujo principal:

Trabajador ingresa los datos del modelo.

El sistema valida y guarda el modelo.

Caso de uso 10: Consultar Modelo

- Actor: Trabajador
- Descripción: Consultar información de los modelos registrados.



- Flujo principal:

Trabajador accede a la lista de modelos.

El sistema muestra los modelos registrados.

Caso de uso 11: Modificar Modelo

- Actor: Trabajador
- Descripción: Actualizar datos de un modelo existente.
- Flujo principal:

Trabajador busca y selecciona el modelo.

Modifica los campos deseados.

El sistema valida los cambios.

Se guardan las modificaciones.

Caso de uso 12: Eliminar Modelo

- Actor: Trabajador
- Descripción: Eliminar un modelo del sistema.
- Flujo principal:

Trabajador selecciona el modelo.

Confirma la eliminación.

El sistema elimina el modelo.

Caso de uso 13: Consultar Clientes

- Actor: Trabajador
- Descripción: Visualizar lista y datos de clientes registrados.



- Flujo principal:

Trabajador accede a Consultar Clientes.

El sistema muestra la lista de clientes.

Caso de uso 14: Consultar Ventas

- Actor: Trabajador
- Descripción: Consultar historial de ventas realizadas.
- Flujo principal:

Trabajador accede al módulo de ventas.

El sistema muestra el historial de ventas.

Caso de uso 15: Iniciar sesión como administrador

- Actor: Admin
- Descripción: Autenticación del administrador.
- Flujo principal:

Admin ingresa usuario y contraseña.

El sistema valida las credenciales y el rol de administrador.

Se concede acceso al panel de administración.

- Flujo alternativo:

Contraseña incorrecta : mensaje de error.

Caso de uso 16: Registrar Trabajador



- Actor: Admin
- Descripción: Crear cuenta para un nuevo trabajador.
- Flujo principal:

Admin ingresa los datos del trabajador.

El sistema crea el usuario (el ID es autoincremental).

Se asigna el rol.

Caso de uso 17: Eliminar Trabajador

- Actor: Admin
- Descripción: Dar de baja a un trabajador.
- Flujo principal:

Admin selecciona el trabajador.

Confirma la eliminación o desactivación.

El sistema procede con la baja.

Caso de uso 18: Ver lista de trabajadores

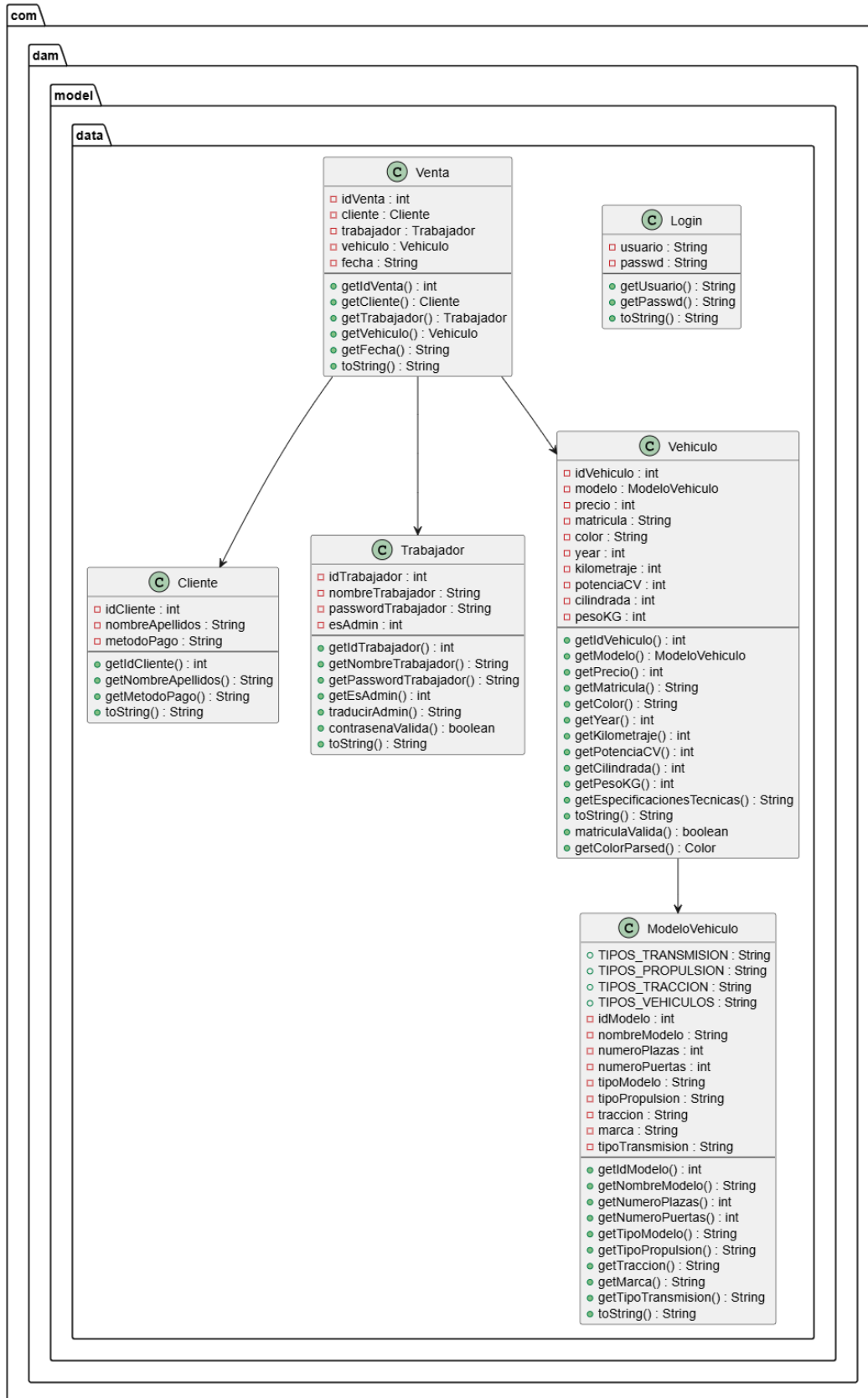
- Actor: Admin
- Descripción: Visualizar todos los trabajadores registrados.
- Flujo principal:

Admin accede a la opción Ver lista de trabajadores.

El sistema muestra la lista con nombre y rol de cada trabajador.



VI. Diagrama de clases



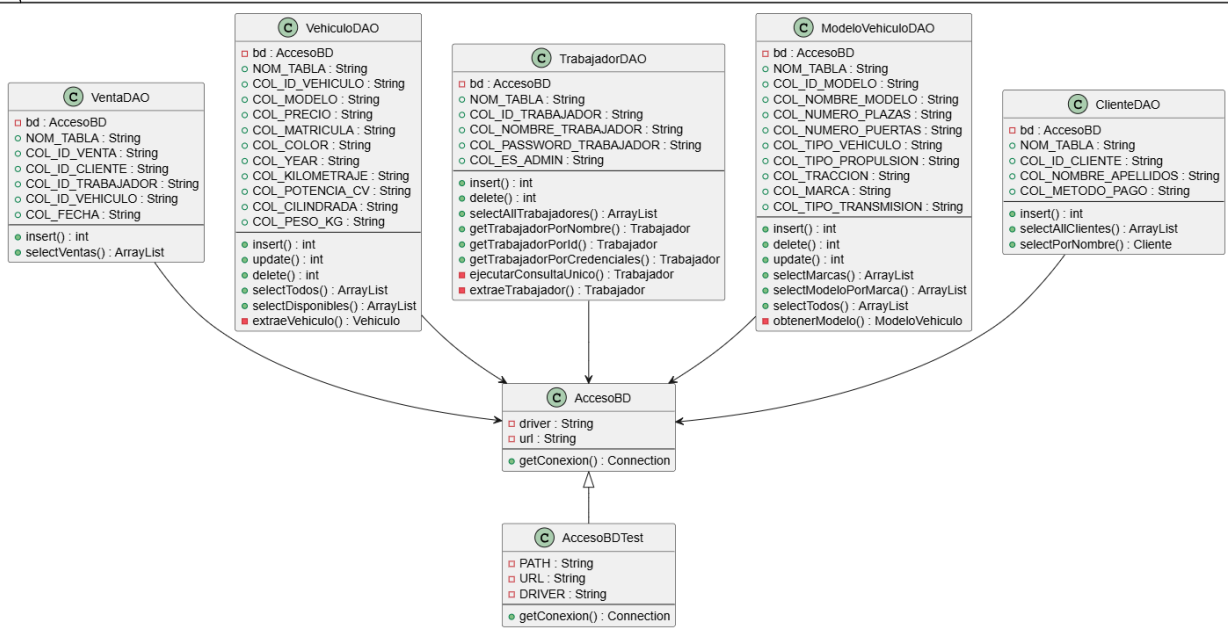


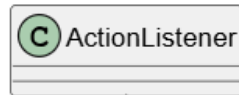
com

dam

model

db





com

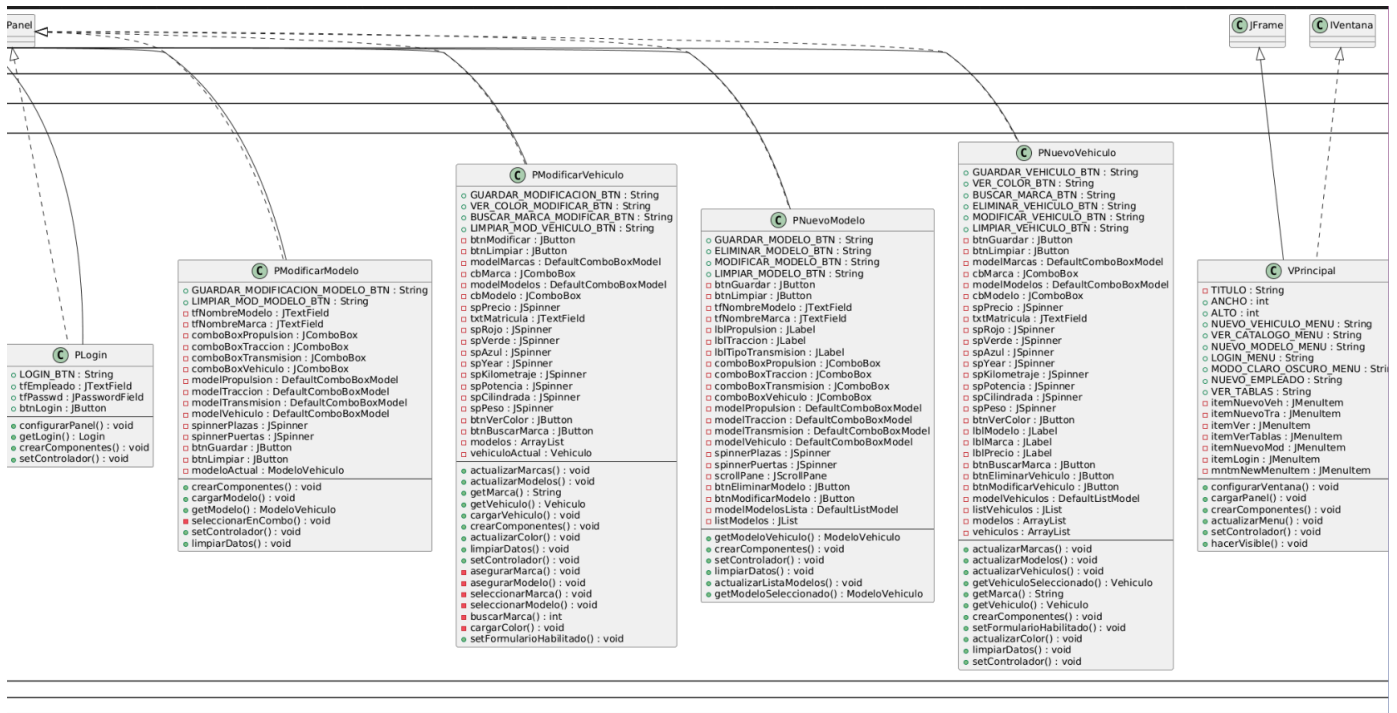
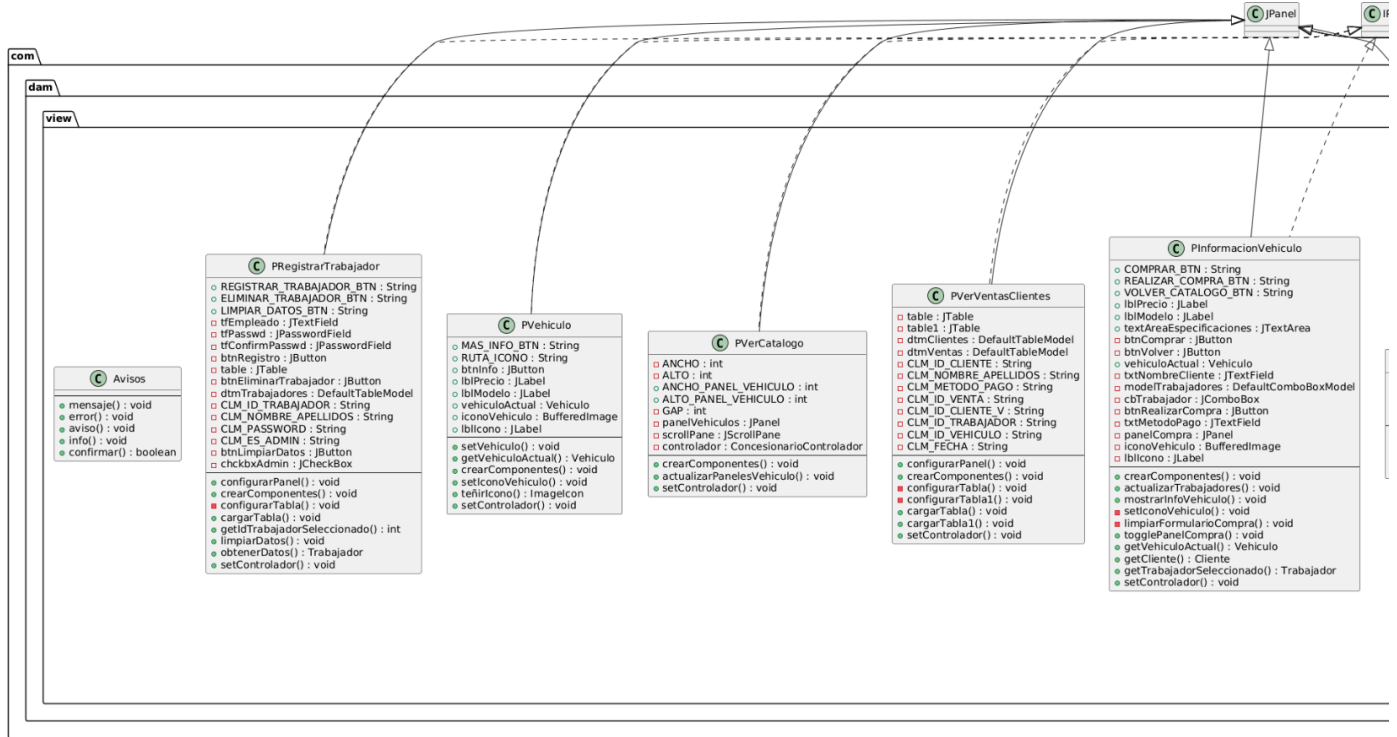
dam

control

 ConcesionarioControlador

- v : VPrincipal
- pNuevoVehiculo : PNuevoVehiculo
- pVerCatalogo : PVerCatalogo
- pNuevoModelo : PNuevoModelo
- pModificarModelo : PModificarModelo
- pModificarVehiculo : PModificarVehiculo
- pLogin : PLogin
- pRegistrarTrabajador : PRegistrarTrabajador
- pInformacionVehiculo : PInformacionVehiculo
- pVerVentasClientes : PVerVentasClientes
- clienteDAO : ClienteDAO
- modeloVehiculoDAO : ModeloVehiculoDAO
- trabajadorDAO : TrabajadorDAO
- vehiculoDAO : VehiculoDAO
- ventaDAO : VentaDAO
- modoClaro : boolean
- sesionIniciada : boolean
- admin : boolean
- DEBUG_ADMIN : boolean

- actionPerformed() : void
- controlMenus() : void
- consultarClientesVentas() : void
- controlBotones() : void
- comprarVehiculo() : void
- cargarPanelCatalogo() : void
- cargarPanelModificarVehiculo() : void
- modificarVehiculo() : void
- eliminarVehiculo() : void
- actualizarModoClaroOscuro() : void
- guardarVehiculo() : void
- guardarModelo() : void
- validarMatriculaVehiculo() : boolean
- cargarPanelModificarModelo() : void
- modificarModelo() : void
- eliminarModelo() : void
- login() : void
- buscarMarca() : void
- buscarMarcaModificar() : void
- consultarTrabajadores() : void
- registrarTrabajador() : void
- eliminarTrabajador() : void



VII. Modelo de datos

1. Cliente

Representa a los compradores.

- **id_cliente** (INTEGER) - *Clave Primaria (Autoincremental)*
- **nombre_apellidos_c** (VARCHAR 100)
- **metodo_pago** (VARCHAR 50)

2. Modelo

Define las características genéricas de un modelo de coche/vehículo.

- **id_modelo** (INTEGER) - *Clave Primaria (Autoincremental)*
- **nombre_modelo** (VARCHAR 80)
- **numero_plazas** (INTEGER)
- **numero_puertas** (INTEGER)
- **tipo_vehiculo** (VARCHAR 100)
- **tipo_propulsion** (VARCHAR 100)
- **traccion** (VARCHAR 20)
- **marca** (VARCHAR 70)
- **tipo_transmision** (VARCHAR 30)

3. Trabajador

Representa a los empleados de la concesionaria.

- **id_trabajador** (INTEGER) - *Clave Primaria (Autoincremental)*
- **nombre_apellidos_t** (VARCHAR 100)
- **password_trabajador** (VARCHAR 100)
- **es_admin** (INTEGER) - *(Actúa como booleano: 1 = admin, 0 = empleado normal)*

4. Vehículo

Representa la unidad física específica que se vende, basada en un modelo.

- **id_vehiculo** (INTEGER) - *Clave Primaria (Autoincremental)*
- **modelo** (INTEGER) - *Clave Foránea hacia Modelo(id_modelo)*
- **precio** (INTEGER)
- **matricula** (VARCHAR 40) - *(Valor Único, no se pueden repetir)*
- **color** (VARCHAR 40)
- **year** (INTEGER)
- **kilometraje** (INTEGER)
- **potencia_cv** (INTEGER)
- **cilindrada** (INTEGER)

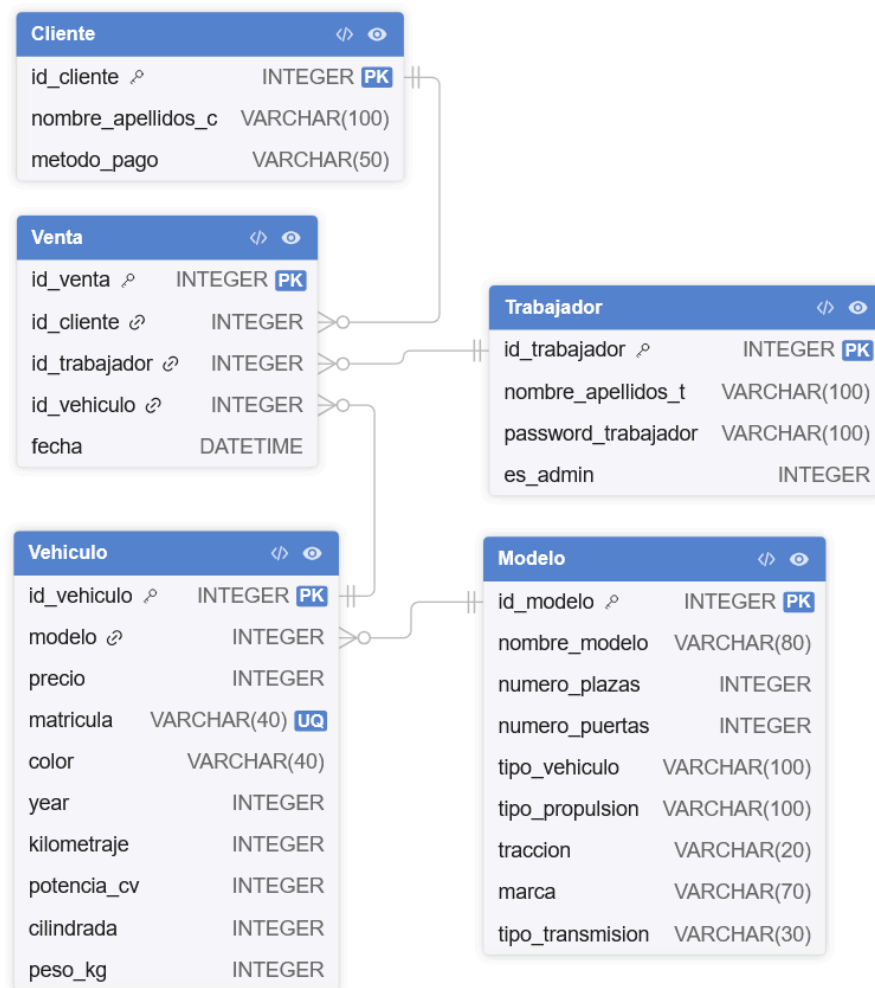


- **peso_kg** (INTEGER)

5. Venta

Registro de la transacción donde un cliente compra un vehículo a través de un trabajador.

- **id_venta** (INTEGER) - *Clave Primaria (Autoincremental)*
 - **id_cliente** (INTEGER) - *Clave Foránea hacia Cliente(id_cliente)*
 - **id_trabajador** (INTEGER) - *Clave Foránea hacia Trabajador(id_trabajador)*
 - **id_vehiculo** (INTEGER) - *Clave Foránea hacia Vehiculo(id_vehiculo)*
 - **fecha** (DATETIME) - *(Por defecto toma la fecha y hora actual del sistema en el momento de la inserción)*
- **Relación con la base de datos**



(Diagrama MER de la base de datos)

Las entidades y atributos del diagrama MER, los trasladamos a un script para crear la base de datos basado en este mismo diagrama, creando tablas de cada entidad y sus elementos.



```
BEGIN TRANSACTION;
DROP TABLE IF EXISTS "Venta";
DROP TABLE IF EXISTS "Vehiculo";
DROP TABLE IF EXISTS "Modelo";
DROP TABLE IF EXISTS "Cliente";
DROP TABLE IF EXISTS "Trabajador";

CREATE TABLE "Cliente" (
    "id_cliente" INTEGER,
    "nombre_apellidos_c" varchar(100),
    "metodo_pago" varchar(50),
    CONSTRAINT "pk_id_cli" PRIMARY KEY("id_cliente" AUTOINCREMENT)
);

CREATE TABLE "Modelo" (
    "id_modelo" INTEGER,
    "nombre_modelo" varchar(80),
    "numero_plazas" INTEGER,
    "numero_puertas" INTEGER,
    "tipo_vehiculo" varchar(100),
    "tipo_propulsion" varchar(100),
    "traccion" varchar(20),
    "marca" varchar(70),
    "tipo_transmision" varchar(30),
    CONSTRAINT "pk_id" PRIMARY KEY("id_modelo" AUTOINCREMENT)
);

CREATE TABLE "Trabajador" (
    "id_trabajador" INTEGER,
    "nombre_apellidos_t" varchar(100),
    "password_trabajador" varchar(100),
    "es_admin" INTEGER,
    CONSTRAINT "pk_id_trabajo" PRIMARY KEY("id_trabajador" AUTOINCREMENT)
);

CREATE TABLE "Vehiculo" (
    "id_vehiculo" INTEGER,
    "modelo" INTEGER,
    "precio" INTEGER,
    "matricula" varchar(40) UNIQUE,
    "color" varchar(40),
    "year" INTEGER,
    "kilometraje" INTEGER,
    "potencia_cv" INTEGER,
    "cilindrada" INTEGER,
    "peso_kg" INTEGER,
    CONSTRAINT "pk_id_car" PRIMARY KEY("id_vehiculo" AUTOINCREMENT),
    CONSTRAINT "fk_model" FOREIGN KEY("modelo") REFERENCES "Modelo"("id_modelo")
);

CREATE TABLE "Venta" (
    "id_venta" INTEGER,
    "id_cliente" INTEGER,
    "id_trabajador" INTEGER,
    "id_vehiculo" INTEGER,
    "fecha" DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "pk_id_venta" PRIMARY KEY("id_venta" AUTOINCREMENT),
    CONSTRAINT "fk_id_cli" FOREIGN KEY("id_cliente") REFERENCES
"Cliente"("id_cliente"),
```



```
CONSTRAINT "fk_id_trabajo" FOREIGN KEY("id_trabajador") REFERENCES
"Trabajador"("id_trabajador"),
CONSTRAINT "fk_id_car" FOREIGN KEY("id_vehiculo") REFERENCES
"Vehiculo"("id_vehiculo")
);
INSERT INTO "Cliente" ("id_cliente","nombre_apellidos_c","metodo_pago") VALUES (2,'Ana
Ruiz López','Tarjeta de crédito'),
(3,'Pedro Infante Rivera','Transferencia bancaria');
INSERT INTO "Modelo"
("id_modelo","nombre_modelo","numero_plazas","numero_puertas","tipo_vehiculo","tipo_prop
ulsion","traccion","marca","tipo_transmision") VALUES (17,'Clase
A',5,4,'Compacto','Gasolina','Delantera','Mercedes-Benz','Automático'),
(18,'Golf',5,4,'Compacto','Diesel','Delantera','Volkswagen','Manual'),
(19,'Model 3',5,4,'Turismo','Eléctrico','AWD','Tesla','Automático'),
(20,'Actros',2,2,'Camión','Diesel','Trasera','Mercedes-Benz','Manual'),
(21,'Civic',5,4,'Turismo','Híbrido','Delantera','Honda','Automático'),
(22,'Ducati 959',2,0,'Motocicleta','Gasolina','Trasera','Ducati','Manual'),
(23,'Moviq',10,2,'Autobús urbano','Eléctrico','Delantera','IVECO','Automático'),
(24,'Transit',3,4,'Furgoneta','Diesel','Delantera','Ford','Manual'),
(25,'Cayenne',5,4,'SUV','Híbrido enchufable','AWD','Porsche','Automático'),
(26,'Mustang',4,2,'Deportivo','Gasolina','Trasera','Ford','Manual'),
(27,'Outlander',7,4,'SUV','Híbrido enchufable','4x4','Mitsubishi','Automático'),
(28,'Hilux',5,4,'Pick-up','Diesel','4x4','Toyota','Manual'),
(29,'Ibiza',5,4,'Compacto','Gasolina','Delantera','SEAT','Manual'),
(30,'Clase S',5,4,'Berlina','Híbrido enchufable','AWD','Mercedes-Benz','Automático'),
(31,'Sprinter',2,4,'Furgoneta','Diesel','Delantera','Mercedes-Benz','Manual');
INSERT INTO "Trabajador"
("id_trabajador","nombre_apellidos_t","password_trabajador","es_admin") VALUES
(85,'Daniel','admin',1),
(86,'Luis','admin',1),
(87,'Empleado','empleado',0);
INSERT INTO "Vehiculo"
("id_vehiculo","modelo","precio","matricula","color","year","kilometraje","potencia_cv",
"cilindrada","peso_kg") VALUES
(17,17,28500,'4521 BKR','#1E1E23',2021,32000,163,1332,1365),
(18,18,19900,'7834 MPT','#F3E5AB',2019,87000,115,1968,1370),
(19,19,41200,'3301 ZFG','#B42828',2022,8000,358,0,1844),
(20,20,89000,'9821 TRK','#556B2F',2020,210000,430,12800,7500),
(21,21,24300,'6612 HNV','#3264A0',2021,45000,143,1498,1395),
(22,22,18700,'1123 DCV','#D2691E',2020,19000,157,955,209),
(23,23,320000,'2255 BUS','#98FF98',2023,5000,240,0,9800),
(24,24,31500,'8847 FGT','#E6E6FA',2018,135000,130,1995,2100),
(25,25,97000,'5593 PKQ','#1E1E23',2022,12000,462,2995,2195),
(26,26,54900,'4478 MST','#3264A0',2020,27000,450,4951,1668),
(27,27,38600,'7765 OTL','#AFEEEE',2021,53000,224,2360,1990),
(28,28,34200,'3349 HLX','#D2B48C',2019,98000,204,2755,2100),
(29,29,14800,'9901 IBZ','#B42828',2020,61000,95,1498,1075),
(30,30,112000,'6630 MBS','#1E1E23',2023,3000,510,2999,2215),
(31,31,42000,'1182 SPR','#FFD1DC',2019,175000,143,2143,3050);
COMMIT;
```

(Script para la creación de la base de datos)

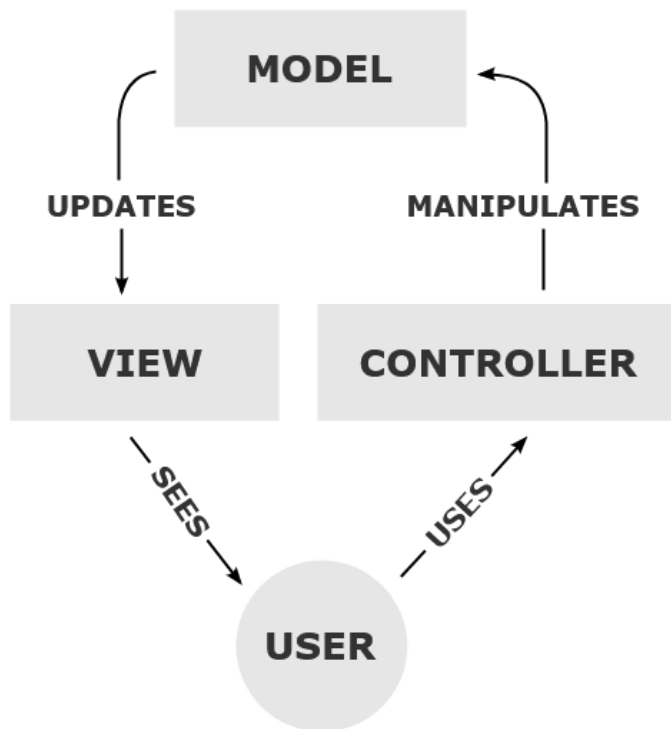
Este es el script que usamos para crear la base de datos, del cual nos hemos basado para la creación del diagrama MER, lo guardamos en un archivo sql y lo trasladamos al proyecto java de la aplicación para poder usar la base de datos en la aplicación.

VIII. Arquitectura de la aplicación

Explicación del patrón MVC

La aplicación se ha desarrollado siguiendo el patrón de diseño **Modelo-Vista-Controlador (MVC)**. Este patrón divide el software en tres componentes conectados para separar la lógica de la interfaz de usuario y de la base de datos.

- **Modelo:** Se encarga de gestionar la información de la aplicación y de interactuar directamente con la base de datos.
- **Vista:** Es la interfaz gráfica de usuario. Su única responsabilidad es mostrar los datos en la pantalla y capturar los eventos del usuario (como pulsar un botón o rellenar un formulario), delegando su procesamiento al componente adecuado.
- **Controlador:** Actúa como intermediario entre la Vista y el Modelo. Recibe las acciones del usuario capturadas por la interfaz, ejecuta la lógica correspondiente y actualiza el Modelo.



Explicación de las tecnologías usadas

- **Lenguaje de programación (Java):** Se ha seleccionado Java como lenguaje principal.
- **Biblioteca gráfica (Java Swing):** La interfaz de usuario se ha desarrollado utilizando la API de Java Swing.
- **Sistema Gestor de Base de Datos (SQLite):** Para la persistencia de los datos se ha optado por SQLite.

IX. Equipo

Nuestro equipo está conformado por:

- Luis Daniel López Milicia.
- Daniel Parra Segovia.
- Hugo García Salazar.

X. Observaciones

- Enlace GitHub del proyecto:

<https://github.com/lunastrod/proyectoIntegradorConcesionario>

- Enlace Jira del proyecto:

<https://danielparrasegovia.atlassian.net/jira/software/projects/SCRUM/boards/1?atlOrigin=eyJpIjoiYjhhMDhiMWRhMWE0NDI2NGEwNTkYzFkZjIjImDU1MjciLCJwIjoiaj9>